ARYA COLLEGE OF ENGINEERING & I.T., JAIPUR

FINAL PROJECT REPORT

KodBTW — Unified Coding Identity & Campus Coding Intelligence Platform

Advanced Java Laboratory / Project Based Learning
Academic Year 2026–27

Project duration: 6 July 2026 – 6 October 2026
Project guide: Er. Ram Babu Buri, Department of CSE
Project code: PBL2627-CSE-C-064

Team Members

Name	Enrollment ID
Sajal Goyal	24E1ARCSM40P142
Shri Krishna Chandra Vaishnav	24E1ARCSM30P157
Yug Mittal	24E1ARCSM40P189
Ravi Shah	24E1ARCSM30P130
Suryadev Singh	24E1ARCSM40P170

Final Project Report

Advanced Java Laboratory / Project Based Learning

Arya College of Engineering & I.T., Jaipur

Academic Year: 2026-27

Project duration: 6 July 2026 to 6 October 2026

Project guide: Er. Ram Babu Buri, Department of CSE

Project team

Name	Enrollment ID
Sajal Goyal	24E1ARCSM40P142
Shri Krishna Chandra Vaishnav	24E1ARCSM30P157
Yug Mittal	24E1ARCSM40P189
Ravi Shah	24E1ARCSM30P130
Suryadev Singh	24E1ARCSM40P170

Project identifier shown in the supplied PBL portal export: PBL2627-CSE-C-064.

Abstract

KodBTW is a web application that brings selected competitive-programming profile data into a user-centered profile and analytics system. The project addresses the inconvenience of viewing accounts and progress separately across online coding platforms while preserving the fact that those platforms expose different statistics and data contracts. The implementation uses a React and TypeScript frontend with a Java 21 / Spring Boot REST backend and MySQL persistence. Users can register and sign in, maintain a KodBTW profile, connect platform handles, view dashboards and historical summaries, consult a leaderboard, and share a public developer profile. Live public-statistics adapters exist for LeetCode and Codeforces; CodeChef, GeeksforGeeks, and HackerRank account records are supported but live statistics synchronization is unavailable. The database records source-tagged, dated snapshots and allows unsupported metrics to remain null. This report documents the software requirements, architecture, modules, database schema, UML, literature context, and verification state. Source inspection confirms the described components, but automated execution and application screenshots were not available in the environment used to prepare this report. The work therefore reports implementation scope and evidence limits, not measured learning impact.

Keywords— competitive programming, Java, Spring Boot, React, learning analytics, data provenance, online judge.

Table of Contents

• Introduction and Background

• Problem Statement, Motivation, Objectives, and Scope

• Existing System and Limitations

• Literature Review and Comparative Analysis

• Research Gap, Questions, and Method

• Software Requirements Specification

• System Architecture and Design

• Module Description and Team Responsibilities

• UML Diagrams

• Database Design and ER Model

• Technology Stack
• Implementation Details
• User Interface and Output Evidence
• Testing Strategy and Verified Results
• Research Findings and Relation to Implementation
• Limitations and Challenges
• Conclusion and Future Scope
• References
Appendices

List of Figures

• Fig. 1. KodBTW component architecture.
• Fig. 2. Current relational ER diagram.
• Fig. 3. User login and protected API sequence.
• Fig. 4. Platform synchronization activity.

List of Tables

• Table I. Project objectives and implementation contribution.
• Table II. Existing and proposed system comparison.
• Table III. Platform source availability.
• Table IV. Major modules and source locations.
• Table V. Database entity summary.
• Table VI. Verification status.

1. Introduction and Background

Competitive programming practice takes place across online judges and coding platforms. Each service maintains its own user profile, rating system, contest records, solved-problem counts, and activity summaries. Learners who use several services must navigate separate interfaces to review those records. A consolidated view can reduce that navigation burden, but only when field meanings and source coverage remain visible.

KodBTW's original proposal describes a unified coding identity and a long-term campus coding-intelligence vision. The current application implements a narrower platform: individual accounts and profiles, platform-account records, supported profile synchronization, persisted statistics, dashboards, a leaderboard, and public profiles. Research on learning analytics dashboards and programming education informs the design context [4]-[12], [16]-[23]. However, this software implementation has not been evaluated with students and makes no causal claim about learning improvement.

1.1 Project particulars

The project is submitted for Advanced Java Laboratory / Project Based Learning, Java Projects, 5th semester, academic year 2026-27, at Arya College of Engineering & I.T., Jaipur. The guide shown in the supplied portal export is Er. Ram Babu Buri of the Department of CSE. The official project period supplied by the user is 6 July 2026 to 6 October 2026. The team roster is listed on the cover from the supplied PBL portal export.

2. Problem Statement, Motivation, Objectives, and Scope

2.1 Problem statement

Coding-platform data is fragmented across services with different available fields and meanings. A user has no single application in this project to manage a consolidated profile, review supported statistics and recorded history, and share a public developer profile. Directly combining raw platform metrics without tracking source and missing-data behavior could produce misleading summaries.

2.2 Motivation and significance

The project applies Java backend concepts, REST APIs, authentication, persistence, schema migrations, and web frontend development to a practical multi-source data problem. The engineering significance is in integrating services while retaining source-specific boundaries and persisted observations. The academic significance is a demonstrable full-stack Advanced Java application, not a validated learning intervention.

2.3 Objectives

• Provide account registration and JWT-based authentication.

• Provide an editable KodBTW profile and a public profile route.

• Store coding-platform handles and synchronize data only from implemented live sources.

• Persist dated, source-tagged statistics for dashboard and historical views.

• Provide analytics, activity/contest summaries, and leaderboard reads based on available persisted data.

• Document database design, security boundaries, current implementation, and verification gaps.

2.4 Scope

The implemented application supports five platform-account types. Live statistics are available in source for LeetCode and Codeforces. Three platform adapters report live sync unavailable. The scope excludes code submission/execution, a contest catalog feed, topic-level DSA classification, account-ownership verification workflow, mentor or campus-administrator dashboards, predictive recommendations, and AI-generated insights.

Table I. Project objectives and contribution

Objective	Implemented contribution
Consolidate coding identity	User profile and platform-account records
Show selected platform activity	Source-specific adapter and persisted stats service
Track change over time	Dated platform snapshots and history services
Compare within the application	Analytics and leaderboard services
Share a developer profile	Public username route and sanitized response DTO
Preserve limitations	Null fields, source labels, and unavailable adapters

3. Existing System and Limitations

3.1 Existing workflow

Without KodBTW, a learner opens each platform separately. This can be sufficient for a single service, but comparing multiple accounts and revisiting observations at different times requires switching systems. The project does not claim that all existing sites lack analytics; each may provide its own platform-specific profile, history, or contest views.

3.2 Proposed improvement and limitations

KodBTW centralizes supported profile data and presents selected metrics through a common application. It cannot eliminate differences in provider definitions, and its current live-source coverage is partial.

Table II. Existing workflow and current system

Aspect	Separate platform workflow	Current KodBTW implementation
Account identity	Managed per external site	Handles stored as user-owned account rows
Metric coverage	Platform-specific	LeetCode and Codeforces live subsets; other sources unavailable
Historical view	Platform-specific, varies	Based on snapshots persisted by KodBTW
Aggregated dashboard	Requires switching between platform pages	Dashboard and analytics endpoints consume eligible saved statistics
Campus monitoring	Not part of this comparison	Not implemented
Recommendations	Provider-specific if present	No AI or DSA-topic recommendations implemented

3.3 Platform availability

Table III. Platform source availability in current code

Platform	Account record	Live stats adapter	Current limitation
LeetCode	Yes	Yes	Query uses publicly reachable but undocumented GraphQL interface; field availability/terms require review
Codeforces	Yes	Yes	Public user.info supplies only mapped rating/max-rating fields; solved counts and history remain null
CodeChef	Yes	No	Sync returns unavailable; no approved stable source is implemented
GeeksforGeeks	Yes	No	Sync returns unavailable; no approved stable source is implemented
HackerRank	Yes	No	Sync returns unavailable; no approved stable source is implemented

4. Literature Review and Comparative Analysis

Learning analytics dashboard studies examine how learner data is selected, displayed, and connected to student or instructor actions. Reviews identify varied audiences and data types, but also report limited theory grounding and gaps between intended outcomes and evaluation measures [4]-[11], [14], [15]. A randomized study in a Java programming course found mixed results: some intermediate online performance changed, but final exam performance and course passing did not [12]. A dashboard should therefore not be described as educationally effective without a suitable evaluation.

Programming education research uses code snapshots, compilation events, course interactions, online judge submissions, and automated feedback [16]-[21], [23]-[28]. These sources can contain richer learning process evidence than public profile metrics. Results based on contest activity or a single course context may not generalize to another population or data source [16], [21]. The project literature register contains 30 scholarly records, an access-status database, and BibTeX references in research/literature-review/.

The detailed standalone synthesis is LITERATURE_REVIEW.md (../../research/literature-review/LITERATURE_REVIEW.md). It identifies source access per record and does not present abstract-only sources as full-text critical evaluations.

5. Research Gap, Questions, and Method

The bounded research gap is an engineering requirement at the intersection of profile aggregation and learning analytics: retain platform provenance and observation time, preserve missingness, and avoid treating every platform statistic as a comparable construct. This gap is derived from the reviewed literature and source audit; it is not a claim that no similar products exist.

The research questions are: (RQ1) how prior learning-analytics and programming-education studies collect and evaluate learner progress; (RQ2) how the current artifact handles source provenance, unavailable metrics, and history; and (RQ3) which proposed features remain outside implementation and what future evidence they require.

The methodology is a structured narrative literature review and a repository artifact audit. The audit inspects controllers, adapters, services, entities, migrations, route definitions, and test sources. No participant study, usability test, benchmark, or learning-outcome experiment was conducted. Details are in RESEARCH_GAP_AND_OBJECTIVES.md (../../research/RESEARCH_GAP_AND_OBJECTIVES.md) and RESEARCH_PAPER.md (../../research/paper/RESEARCH_PAPER.md).

6. Software Requirements Specification

The separate SRS (../requirements/SRS.md) defines 16 functional requirements and nine non-functional requirements, with source paths, test classes where present, and verification statuses. Requirements cover authentication, profiles, owner-scoped platform accounts, source-limited sync, snapshots, analytics/history, leaderboards, public profiles, security, schema management, and error handling. Requirements for campus roles, topic analytics, verification, and recommendations are explicitly identified as proposed/out of current scope.

7. System Architecture and Design

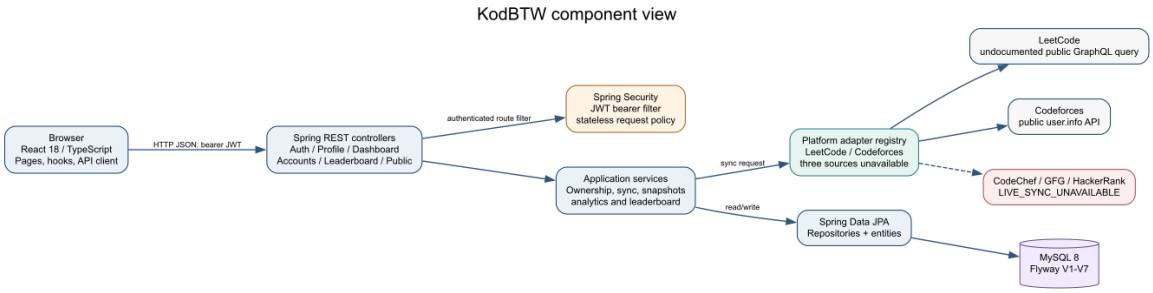


Figure: component diagram. Source: repository code and schema audit.

The frontend calls a Spring REST API. Spring Security checks bearer JWTs. Controllers delegate to services for validation, ownership checks, platform synchronization, data aggregation, and public-profile mapping. Platform adapters isolate provider-specific requests. JPA repositories access MySQL; Flyway manages schema versions.

Fig. 1. KodBTW component architecture. The diagram is available at component diagram (../diagrams/uml/component-diagram.svg). It distinguishes live public sources and adapters marked unavailable.

The design uses a source-tagged snapshot rather than a global, unqualified metric row. Dashboard, history, and analytics services operate on persisted observations. The leaderboard cache is a denormalized projection intended for paginated reads. Current services do not create historical points from a present-day value.

8. Module Description and Team Responsibilities

Table IV. Major modules and source locations

Module	Source location	Objective contribution
Authentication/security	backend/src/main/java/com/kodbtw/{controller,service,security,config}	Registration, login, JWT filtering, protected routes

Module	Source location	Objective contribution
Profiles	Backend profile/public-profile controllers, services, DTOs; frontend profile pages	Private profile edit/read and public profile display
Platform-account management	PlatformAccountController, PlatformAccountService, PlatformAccount	User-owned coding-handle CRUD and sync status
Platform adapters	backend/src/main/java/com/kodbtw/adapter/	Provider request/response mapping and explicit unsupported-platform behavior
Statistics and history	PlatformSyncService, persistence service, stats/history services	Sync lifecycle, snapshots, current data, history
Dashboard analytics	Analytics, insights, activity/contest services and frontend components	Summaries and comparison views from stored data
Leaderboard	Controller, leaderboard services, cache entity	Paginated ranking reads and cache rebuild path
Frontend	frontend/src/pages, components, hooks, services/api	User-facing screens and typed API interactions
Persistence/migrations	entity, repository, Flyway V1-V7	Relational data and schema evolution

The portal export confirms five team members but shows zero team progress and no submitted daily or weekly logs. The shallow Git checkout exposes one commit by Sajal Goyal. These are not sufficient to attribute all module work to named members. Accordingly, the team contribution document lists member-to-module assignments as unverified rather than inventing them. See TEAM_CONTRIBUTIONS.md (../TEAM_CONTRIBUTIONS.md).

9. UML Diagrams

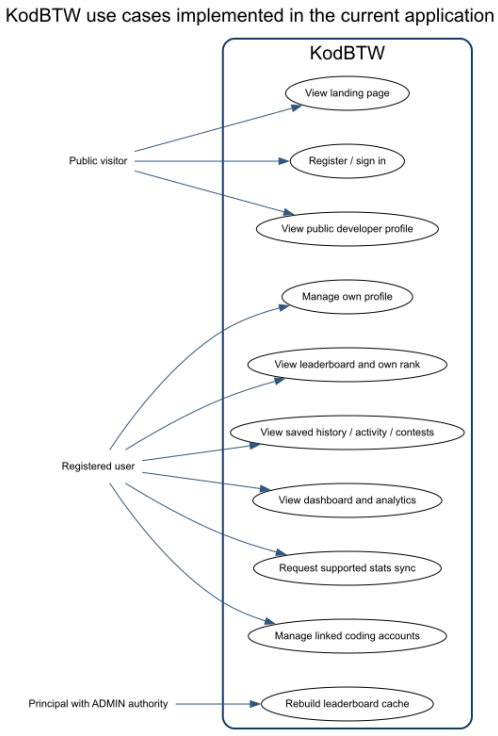


Figure: use case diagram. Source: repository code and schema audit.

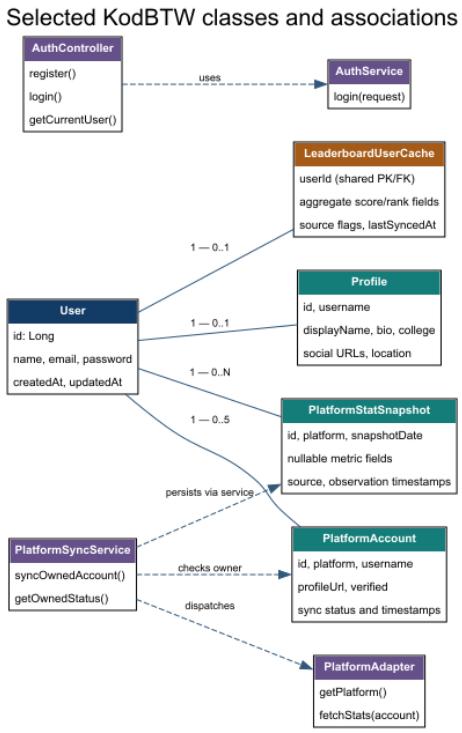


Figure: class diagram. Source: repository code and schema audit.

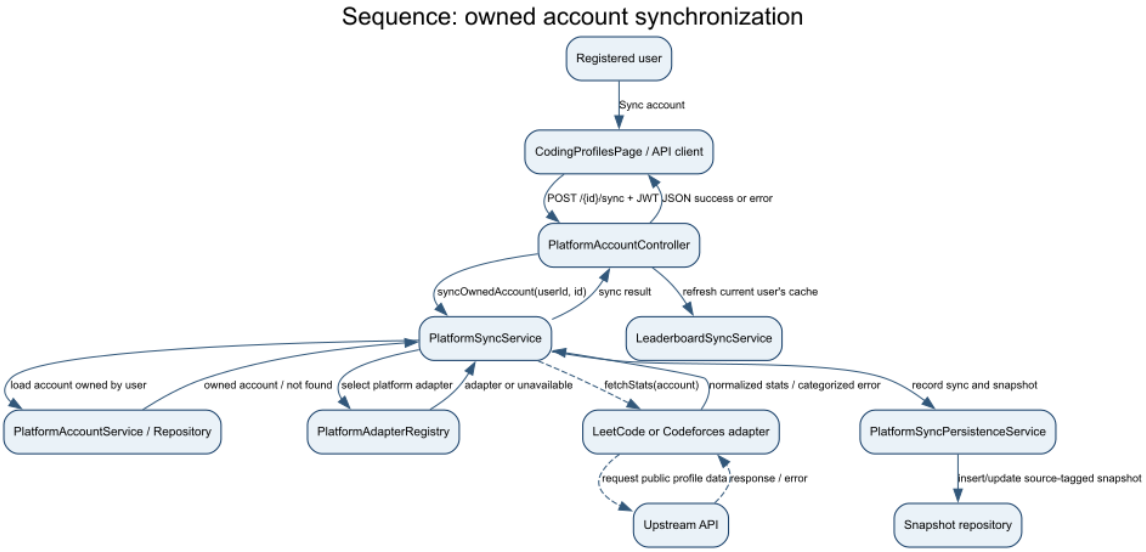


Figure: sequence platform sync. Source: repository code and schema audit.

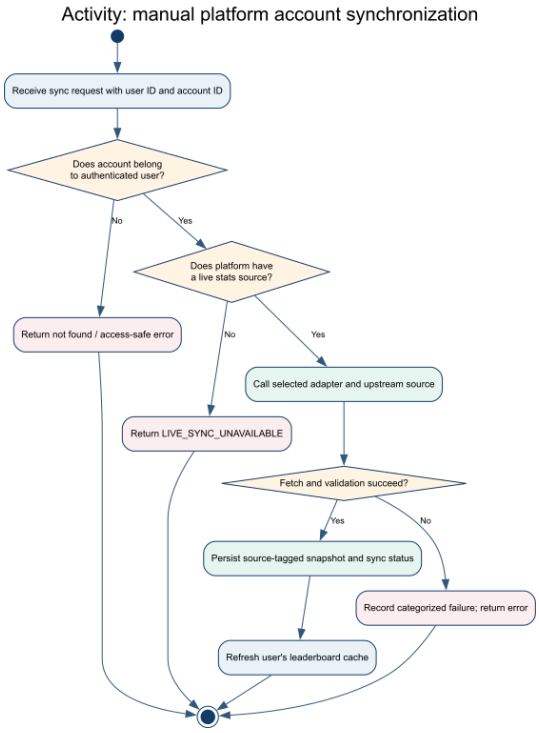


Figure: activity platform sync. Source: repository code and schema audit.

The diagrams are generated from editable Graphviz DOT source and describe the inspected implementation:

• Use-case diagram: public visitor, registered user, and ADMIN-authority rebuild.

• Class diagram: selected JPA entities and controller/service dependencies.

• Component diagram: frontend, API, security, adapters, persistence, and upstream sources.

• Login sequence and platform-sync sequence.

• Sync activity flow and intended deployment topology.

Each source, SVG, PNG, and explanation is indexed in the UML diagram index (../diagrams/uml/INDEX.md). The diagrams do not show proposed mentor/campus roles.

10. Database Design and ER Model

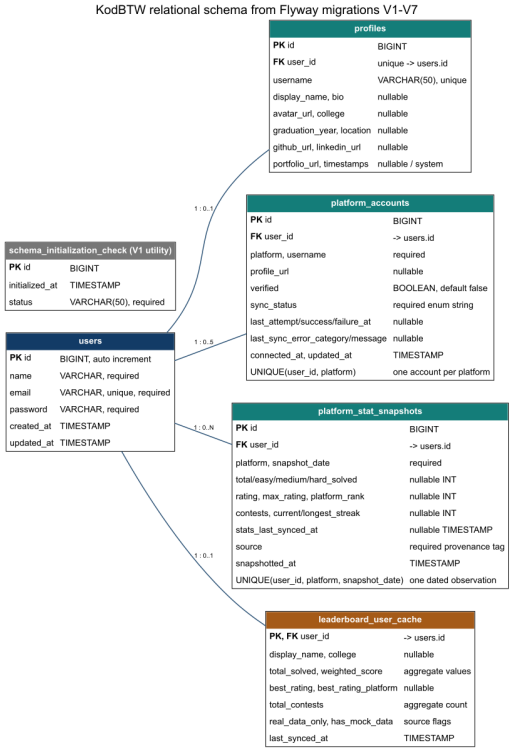


Figure: ER DIAGRAM. Source: repository code and schema audit.

The project uses MySQL 8 and seven Flyway migrations. Hibernate validates the schema rather than creating or updating it. Major tables are users, profiles, platform_accounts, platform_stat_snapshots, and leaderboard_user_cache; V1 also creates an isolated schema-initialization utility table.

Table V. Database entities

Entity	Relationship and purpose
users	Root identity; unique email
profiles	Zero or one per user; unique public username
platform_accounts	Zero to five per user; unique (user_id, platform)
platform_stat_snapshots	Many observations per user; unique (user_id, platform, snapshot_date)
leaderboard_user_cache	Zero or one rebuildable projection per user
schema_initialization_check	Migration marker with no domain relationship

Fig. 2. Current ER diagram. The exported diagram and schema notes are in the database documentation folder (../diagrams/database/).

The account and snapshot foreign keys cascade on user deletion. Snapshot metric fields remain nullable because sources differ in available data. The source column provides provenance, and the uniqueness constraint prevents duplicate daily snapshots per user/platform.

11. Technology Stack

Layer	Technology	Use
Frontend	React 18, TypeScript, Vite	SPA routes, forms, dashboards, typed client
Backend	Java 21, Spring Boot 3.3, Spring Web	REST controllers and services
Security	Spring Security, JWT, password encoder	Stateless authentication and protected routes
Persistence	Spring Data JPA, Hibernate	Entity mapping and repositories
Database	MySQL 8, Flyway	Relational storage and versioned migrations
Testing	Spring Boot Test, H2, Spring Security Test	Backend test source suite
Build	Maven wrapper, npm	Backend/frontend build entry points

12. Implementation Details

12.1 Authentication and profiles

Registration and login are exposed through AuthController. AuthService normalizes email, validates credentials using a password encoder, and requests a JWT from JwtService. The API also exposes /api/auth/me. A profile is linked one-to-one to a user and has a unique public username. Public profile output is served by a separate public controller/service and response DTOs.

12.2 Platform accounts and synchronization

The authenticated user can create, read, update, and delete platform account records. Sync requests pass the current user's ID and account ID to an ownership-aware service. The service selects a platform adapter, records sync lifecycle status, saves a successful source-tagged snapshot, and refreshes the user's leaderboard cache. If the platform has no live source, sync is explicitly unavailable.

12.3 Analytics and leaderboard

Dashboard API endpoints include stats, analytics, history, insights, activity, and contest-intelligence summaries. Services use persisted data and return null/absent values where metrics do not exist. The leaderboard API supports global/college-filtered reads, current-user rank, pagination, and sorting. Manual full cache rebuild is intended for an admin-authorized principal.

12.4 Runtime configuration

Backend runtime requires DB_URL, DB_USERNAME, DB_PASSWORD, and JWT_SECRET. Production CORS origins are configured explicitly. The frontend VITE_API_BASE_URL is public build-time configuration and must not contain secrets. No hosting account or production URL is included as verified evidence.

13. User Interface and Output Evidence

The frontend route source includes landing, login, registration, dashboard, analytics, contests, leaderboard, profile, coding-profile management, and public profile pages. These route definitions establish implemented UI entry points but do not establish that a screen rendered successfully.

No genuine screenshot of a running KodBTW application could be captured in the available environment. The supplied three-page PDF contains screenshots from the Arya College PBL portal, not KodBTW; its daily log and weekly-report pages show no submissions. Those images are not presented as application output. The screenshot index records this limitation and lists the screens to capture after the runtime environment is available: Screenshot index (../screenshots/INDEX.md).

14. Testing Strategy and Verified Results

The repository includes tests for authentication, profiles, platform-account controllers, adapters, sync services, dashboard analytics/history/insights, public profiles, leaderboards, CORS, error handling, and migration behavior. The intended validation sequence is documented in TESTING_AND_VALIDATION.md (../TESTING_AND_VALIDATION.md).

Table VI. Verification status

Check	Outcome in this preparation environment
Backend mvnw.cmd test	Not executed; Maven wrapper reported JAVA_HOME not found
Frontend npm ci	Not executed; npm command unavailable
Frontend production build	Not reached
Live application smoke test	Not run; Java and configured MySQL unavailable
MySQL Flyway validation	Not run; no configured database service/credentials
UI screenshot capture	Not run; app could not be launched
Source/schema audit	Completed by reading files and migrations

Test source presence is not a passing test result. No pass count or test outcome is claimed.

15. Research Findings and Relationship to the System

The literature review indicates that dashboards need to be evaluated against intended learner outcomes and that programming-process studies often rely on richer data than public profile summaries [9]-[12], [16]-[21]. KodBTW's source audit provides an implementation-level finding: supported statistics are tagged and persisted by date, unavailable metrics remain null, and three platform sync paths explicitly fail as unavailable. This aligns with data-integrity concerns, but it does not establish learning benefit.

The accompanying IEEE-format research paper, RESEARCH_PAPER.md (../../research/paper/RESEARCH_PAPER.md), frames the artifact audit and related evidence as a design study. It and this report use the same objectives, source boundaries, citations, and test limitations.

16. Limitations and Challenges

• Upstream platforms expose different metrics and can change their interfaces.

• Codeforces user.info does not provide solved totals or full history.

• The LeetCode GraphQL query is undocumented; terms and stability require review.

• CodeChef, GeeksforGeeks, and HackerRank live statistics are unavailable in current code.

• Cross-platform ratings and counts are not a validated common scale.

• There is no user study, production deployment verification, or runtime evidence in this report.

• The supplied portal export contains no progress log or weekly report entries; member contributions cannot be assigned from the shallow history.

17. Conclusion and Future Scope

KodBTW demonstrates a full-stack Java application for coding identity, supported platform statistics, dashboards, persisted history, leaderboard reads, and public profiles. Its current data model preserves source labels and dates, while nullable fields and explicit unavailable adapters communicate coverage limits. The implementation must not be confused with the proposal's wider campus-intelligence vision.

Next steps are to run the test/build/migration suite on a configured toolchain, capture genuine application screenshots, validate platform terms and API contracts, complete member-confirmed contribution records, and evaluate usability.

Topic analytics, mentor views, institution-wide monitoring, recommendations, and AI features remain future work requiring new requirements and evidence.

18. References

Reference numbering follows the evidence register in research/literature-review/literature_review.csv and the matching BibTeX database. Records with abstract/metadata-only access are used at that evidence level, as disclosed in the literature review.

[1] Z. Papamitsiou, A. A. Economides, ‘Learning Analytics and Educational Data Mining in Practice: A Systematic Literature Review of Empirical Evidence,’ Educational Technology & Society 17(4), 49-64, 2014. https://eric.ed.gov/?id=EJ1045537.

[2] K. Verbert, E. Duval, J. Klerkx, S. Govaerts, J. L. Santos, ‘Learning analytics dashboard applications,’ American Behavioral Scientist 57(10), 1500-1509, 2013. https://doi.org/10.1177/0002764213479363.

[3] K. Verbert, S. Govaerts, E. Duval, J. L. Santos, F. Van Assche, G. Parra, J. Klerkx, ‘Learning dashboards: an overview and future research opportunities,’ Personal and Ubiquitous Computing 18(6), 1499-1514, 2014. https://doi.org/10.1007/s00779-013-0751-2.

[4] B. Schwendimann et al., ‘Perceiving Learning at a Glance: A Systematic Literature Review of Learning Dashboard Research,’ IEEE Transactions on Learning Technologies 10(1), 30-41, 2017. https://doi.org/10.1109/TLT.2016.2599522.

[5] R. Bodily, K. Verbert, ‘Review of Research on Student-Facing Learning Analytics Dashboards and Educational Recommender Systems,’ IEEE Transactions on Learning Technologies 10(4), 405-418, 2017. https://doi.org/10.1109/TLT.2017.2740172.

[6] I. Jivet, M. Scheffel, M. Specht, H. Drachsler, ‘License to evaluate: preparing learning analytics dashboards for educational practice,’ Proceedings of LAK 2018, 31-40, 2018. https://doi.org/10.1145/3170358.3170421.

[7] R. G. Bodily, T. K. Ikahihifo, B. Mackley, C. R. Graham, ‘The design, development, and implementation of student-facing learning analytics dashboards,’ Journal of Computing in Higher Education 30(3), 572-598, 2018. https://doi.org/10.1007/s12528-018-9186-0.

[8] C. Schumacher, D. Ifenthaler, ‘Features students really expect from learning analytics,’ Computers in Human Behavior 78, 397-407, 2018. https://doi.org/10.1016/j.chb.2017.06.030.

[9] W. Matcha, N. A. Ahmad Uzir, D. Gaševi■, A. Pardo, ‘A Systematic Review of Empirical Studies on Learning Analytics Dashboards: A Self-Regulated Learning Perspective,’ IEEE Transactions on Learning Technologies 13(2), 226-245, 2020. https://doi.org/10.1109/TLT.2019.2916802.

[10] N. Valle, P. Antonenko, K. Dawson, A. C. Huggins-Manley, ‘Staying on target: A systematic literature review on learner-facing learning analytics dashboards,’ British Journal of Educational Technology 52, 1724-1748, 2021. https://doi.org/10.1111/bjet.13089.

[11] G. Ramaswami, T. Susnjak, A. Mathrani et al., ‘Use of Predictive Analytics within Learning Analytics Dashboards: A Review of Case Studies,’ Technology, Knowledge and Learning 28, 959-980, 2023. https://doi.org/10.1007/s10758-022-09613-x.

[12] J. Hellings, C. Haelermans, ‘The effect of providing learning analytics on student behaviour and performance in programming: a randomised controlled experiment,’ Higher Education 83, 1-18, 2022. https://doi.org/10.1007/s10734-020-00560-z.

[13] M. Zhu, A. R. Sari, M. M. Lee, ‘Trends and Issues in MOOC Learning Analytics Empirical Research: A Systematic Literature Review (2011-2021),’ Education and Information Technologies 27, 10135-10160, 2022. https://doi.org/10.1007/s10639-022-11031-6.

[14] L. Paulsen, E. Lindsay, ‘Learning analytics dashboards are increasingly becoming about learning and not just analytics - A systematic review,’ Education and Information Technologies 29, 14279-14308, 2024.

https://doi.org/10.1007/s10639-023-12401-4.

[15] O. Viberg, M. Hatakka, O. Bälter, A. Mavroudi, ‘The current landscape of learning analytics in higher education,’ Computers in Human Behavior 89, 98-110, 2018. https://doi.org/10.1016/j.chb.2018.07.027.

[16] P. Ihantola et al., ‘Educational data mining and learning analytics in programming: Literature review and case studies,’ Proceedings of ITiCSE Working Group Reports, 41-63, 2016. https://doi.org/10.1145/2858796.2858798.

[17] P. Blikstein, ‘Using learning analytics to assess students' behavior in open-ended programming tasks,’ Proceedings of the 1st International Conference on Learning Analytics and Knowledge, 110-116, 2011. https://doi.org/10.1145/2090116.2090132.

[18] P. Blikstein, ‘Programming Pluralism: Using Learning Analytics to Detect Patterns in the Learning of Computer Programming,’ Journal of the Learning Sciences 23(4), 2014. https://doi.org/10.1080/10508406.2014.954750.

[19] A. Altadmri, N. C. C. Brown, ‘37 Million Compilations: Investigating Novice Programming Mistakes in Large-Scale Student Data,’ Proceedings of SIGCSE 2015, 522-527, 2015. https://doi.org/10.1145/2676723.2677258.

[20] S. López-Pernas, M. Saqr, O. Viberg, ‘Putting It All Together: Combining Learning Analytics Methods and Data Sources to Understand Students' Approaches to Learning Programming,’ Sustainability 13(9), 4825, 2021. https://doi.org/10.3390/su13094825.

[21] J. Wang, P. Lin, Z. Tang, S. Chen, ‘How problem difficulty and order influence programming education outcomes in online judge systems,’ Heliyon 9(11), e20947, 2023. https://doi.org/10.1016/j.heliyon.2023.e20947.

[22] K. K. F. Yuen, D. Y. W. Liu, H. V. Leong, ‘Competitive programming in computational thinking and problem solving education,’ Computer Applications in Engineering Education 31, 850-866, 2023. https://doi.org/10.1002/cae.22610.

[23] A. Kurnia, A. Lim, B. Cheang, ‘Online judge,’ Computers & Education 36(4), 299-315, 2001. https://doi.org/10.1016/S0360-1315(01)00018-5.

[24] H. Keuning, J. Jeuring, B. Heeren, ‘A systematic literature review of automated feedback generation for programming exercises,’ ACM Transactions on Computing Education 19(1), 2018. https://doi.org/10.1145/3231711.

[25] A. Luxton-Reilly et al., ‘Introductory Programming: A Systematic Literature Review,’ Proceedings Companion of ITiCSE 2018, 2018. https://doi.org/10.1145/3293881.3295779.

[26] I. Araya et al., ‘Predicting student performance in computing courses based on programming behavior,’ Computer Applications in Engineering Education, 2022. https://doi.org/10.1002/cae.22519.

[27] U. Omer, R. Tehseen, M. S. Farooq, A. Abid, ‘Learning analytics in programming courses: Review and implications,’ Education and Information Technologies, 2023. https://doi.org/10.1007/s10639-023-11611-0.

[28] J.-H. Yu, X.-Z. Chang, W. Liu, Z. Huan, ‘An online integrated programming platform to acquire students' behavior data for immediate feedback teaching,’ Computer Applications in Engineering Education 31(3), 520-536, 2023. https://doi.org/10.1002/cae.22596.

[29] W.-Y. Chang, ‘The Effects of Online Judge System on Motivation and Thinking in Programming Education: Structural Relationships between Factors,’ Journal of Korean Association of Computer Education 24(5), 1-16, 2021. https://doi.org/10.32431/kace.2021.24.5.001.

[30] G. Rodrigues, A. F. Monteiro, A. Osório, ‘Introductory Programming in Higher Education: A Systematic Literature Review,’ Third International Computer Programming Education Conference (ICPEC 2022), OASIcs 102, 4:1-4:17, 2022. https://doi.org/10.4230/OASIcs.ICPEC.2022.4.